Module 6 — Architecture & Verification

Docker Compose Architecture

The application runs as four containerized services defined in docker-compose.yml, all connected via Docker Compose's default private network:

- **web** — This is the Flask application. It publishes task messages to RabbitMQ when buttons are clicked.
- **worker** — This is the python consumer. It connects to RabbitMQ on startup and listens indefinitely for tasks
- **db** — Stores applicant records in the applicants table and tracks scraping progress in ingestion_watermarks.
- **rabbitmq** – This provides the tasks direct exchange and tasks_q durable queue

The health checks on db and rabbitmq ensure web and worker only start after both dependencies are confirmed healthy.

RabbitMQ Message Flow

When a user clicks a button on the Flask interface, the corresponding route calls publish_task() in publisher.py. This function opens a connection to RabbitMQ using the RABBITMQ_URL environment variable and then publishes a compact JSON message containing the task kind (e.g. scrape_new_data). Once the message is published, the connection is closed and Flask immediately returns 202 Accepted to the browser. The user sees a "queued" confirmation without waiting for any work to complete.

The worker processes one message at a time to avoid overloading the database. When a message arrives, the worker parses the JSON text, looks up the appropriate handler in its task map, and opens a new PostgreSQL connection. If the job succeeds, the worker commits and sends a basic_ack to RabbitMQ, permanently removing the message from the queue. If things fail, the worker rolls back the transaction and sends a basic_nack with requeue=False, discarding the message rather than retrying it over again. The database connection is then closed.

Database Initialization

The applicants table and ingestion_watermarks table are created on first use via CREATE TABLE IF NOT EXISTS in load_data.py. This runs automatically inside the worker's handle_recompute_analytics handler when "Create Database" is clicked. This table records the last successfully scraped record per source, enabling idempotent incremental scraping; only new records than last_seen are fetched on subsequent runs. All inserts use ON CONFLICT (p_id) DO NOTHING to prevent duplicates.

Worker Behavior

On startup, consumer.py connects to RabbitMQ via RABBITMQ_URL. For each message, the consumer parses the JSON text, opens up a connection to PostgreSQL, and runs the handler. With every success, the worker commits and acks the message. With every failure, the worker ancks without requeue. The worker then closes the connection to the database.

Verification Steps

The following steps were used to verify the containerization.

1. Run docker compose up from *module_6/*
2. Confirm all 4 containers start healthy in Docker Desktop
3. Go to http://localhost:8080 to verify the Flask app is loaded
4. Go to http://localhost:15672 (guest/guest) to view the RabbitMQ management interface
5. Click **Create Database**: the worker logs show recompute_analytics complete
6. Click **Pull Data**: the worker logs show scrape_new_data received and processed
7. Click **Update Analysis**: the UI populates with real GradCafe applicant data
8. Observe delivered messages in the RabbitMQ management UI